

Testing Policy

Version: 2

Last Updated: 29 September 2026



Prepared by



1. Introduction	2
1.1 Purpose	2
1.2 Scope	2
1.3 Standards	2
2. Testing Types	2
2.1 Unit Testing	2
2.2 Integration Testing	2
2.3 Regression Testing	3
2.4 End-to-End (E2E) Testing	3
3. Tools and Environments	3
3.1 Development	3
3.2 CI Environment	3
3.3 Production	4
3.4 Testing Tools	4
4. Acceptance Criteria	5
5. Roles and Responsibilities	5

1. Introduction

1.1 Purpose

This policy sets out the rules and approach for software testing across the Momently project. It defines the objectives, responsibilities, standards, tools, environments and procedures used to verify that the system is stable and correct.

1.2 Scope

This policy applies to all of Momently: the Angular frontend, the Spring Boot backend and the FastAPI AI service. It covers new features, refactoring, bug fixes, security updates, dependency upgrades and deployment changes.

1.3 Standards

- Test early and continuously.
- Every Pull Request must pass the automated tests before it is merged.
- Failing tests and errors must be fixed before merging.
- Regression testing is required after major changes.
- Production deployments use only successful pipeline builds.

2. Testing Types

2.1 Unit Testing

Purpose: To verify that individual classes, functions, components and modules work correctly in isolation. Unit tests are required for the frontend, backend and AI service. External dependencies are replaced with mocks or mock data.

Unit tests follow the **Arrange–Act–Assert (AAA)** pattern:

- **Arrange:** set up the isolated component under test (the system under test, or SUT) and its inputs.
- **Act:** run the SUT with a specific test condition.
- **Assert:** check that the actual outcome matches the expected behaviour.

2.2 Integration Testing

Purpose: To verify that components communicate correctly, so that modules, APIs and the database work together as a group.

The project uses a **sandwich approach**, which combines bottom-up and top-down testing:

- **Backend (bottom-up):** tests start at the database layer and work upwards to the API controllers, using a real PostgreSQL instance in a test container.
- **Frontend (top-down):** tests start at the UI and work downwards, with external network boundaries mocked.

2.3 Regression Testing

Purpose: To confirm that new changes do not break existing functionality or cause unexpected side effects. The full automated suite must be run after:

- bug fixes
- feature enhancements
- dependency upgrades
- Docker image updates

2.4 End-to-End (E2E) Testing

Purpose: To test the application flow from start to finish by simulating real user scenarios and confirming that all subsystems work together. Scenarios are derived from the user stories of the current sprint.

3. Tools and Environments

3.1 Development

Developers must run the relevant tests and validate their changes before pushing to the repository, so errors are caught as early as possible. The system runs locally with Docker Compose, so Docker and Docker Compose must be installed. DBeaver is recommended as a GUI for navigating PostgreSQL.

3.2 CI Environment

GitHub Actions runs a separate pipeline for the backend, frontend and AI service. Each pipeline automates the following:

- building and testing the component
- linting where configured (Ruff for the AI service)
- vulnerability scanning of the built container image (Trivy)
- building the Docker image and pushing it to AWS ECR

This gives consistent builds, faster feedback and less manual work.

3.3 Production

The production environment runs the deployed application on AWS EC2 as a containerised Docker Compose stack. The [deploy.yml](#) workflow runs on every push to [main](#). It pulls the latest images from ECR, restarts the containers and runs a health check. A deployment is only successful if this workflow passes.

3.4 Testing Tools

Activity	Tool
Frontend unit tests	Jest
Frontend E2E tests	Cypress/Playwright
Frontend linting	ESLint
Backend unit tests	JUnit 5
Backend mocking	Mockito
Backend integration tests	Spring Boot Test, Testcontainers, REST Assured
Backend linting	Spring boot spotless
AI service tests	Pytest
AI service linting	Ruff
API testing	Swagger UI/ cURL
Code quality	SonarQube
Security scanning	Zap
Container testing	Docker

CI/CD	GitHub Actions
-------	----------------

4. Acceptance Criteria

A feature is accepted only if:

- its functional requirements are satisfied
- unit and integration tests pass
- the CI/CD pipeline completes successfully
- the Docker containers build and deploy successfully
- it belongs to the current sprint and has been approved by a reviewer through a Pull Request

5. Roles and Responsibilities

- **Developers** implement features, write unit and integration tests, test locally before pushing, and fix defects.
- **Reviewers** check Pull Requests for correctness and test coverage, and approve them only when the pipeline passes.